LAPORAN
PERANCANGAN CLASS DIAGRAM, BASIS DATA, DAN MODUL LOGIN S-TIDY

Ditujukan untuk menyelesaikan tugas ke-6 Implementasi dan Pengujian Perangkat Lunak



Ditulis Oleh:

Raissa Putri Athaya	103012400255
Baliana Daniswara	103012400363
Husnul Khotimah	103012430019
Surya Mirfattul Jannah	103012400372

S1 INFORMATIKA

TELKOM UNIVERSITY

BANDUNG

2025/2026

DAFTAR ISI

DAFTAR ISI **1**

BAB I

PERANCANGAN DAN ANALISIS CLASS DIAGRAM **1**

BAB II

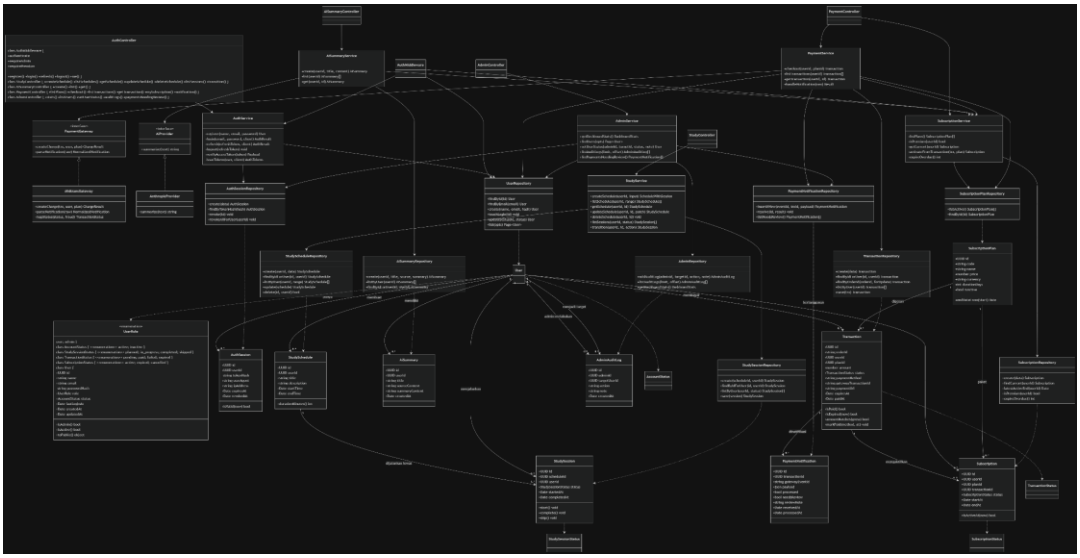
IMPLEMENTASI KODE PROGRAM **1**

LAMPIRAN **2**

BAB I
PERANCANGAN DAN ANALISIS CLASS DIAGRAM

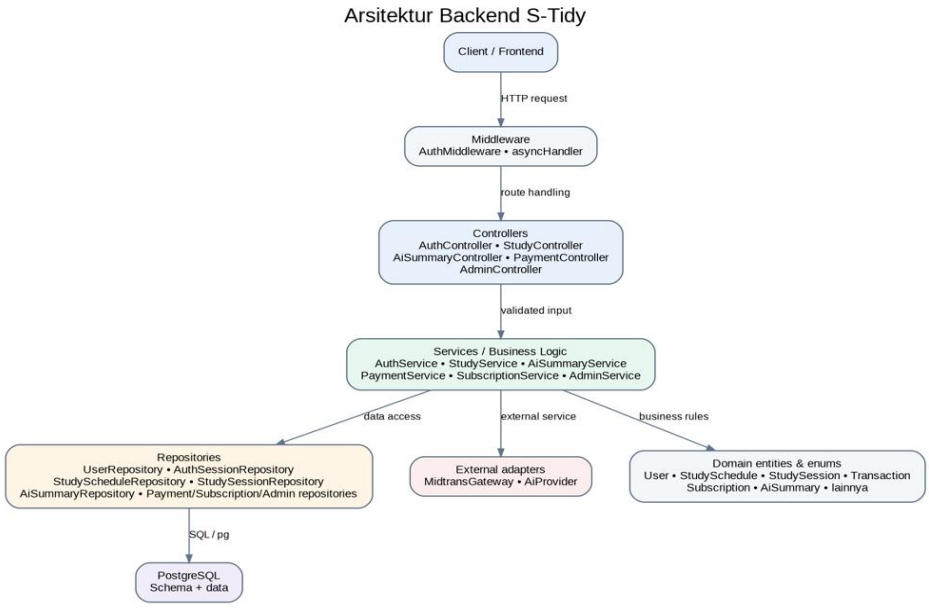
1.1 UML Class Diagram Sistem S-Tidy

S-Tidy merupakan aplikasi perencanaan belajar yang menyediakan pengelolaan jadwal dan sesi belajar, ringkasan berbantuan AI, pembayaran paket premium, serta fungsi administrasi. Berikut adalah UML Class Diagram S-tidy.



Gambar 1. UML Class Diagram S-Tidy

Backend pada repository menggunakan Node.js, TypeScript, Express, dan PostgreSQL. Pola arsitektur utama adalah Controller, Service, Repository, PostgreSQL.



Gambar 2. Arsitektur lapisan backend S-Tidy

1.2 Spesifikasi Detail Kelas (Atribut, Method, Tipe Data, dan Access Modifier)

Tabel berikut disusun dari deklarasi TypeScript pada src/domain/entities.ts dan src/domain/enums.ts. Dalam TypeScript, anggota kelas yang tidak diberi modifier secara eksplisit bersifat public. Tanda ! pada atribut, misalnya id!: string, adalah definite assignment assertion; tanda itu bukan access modifier. Atribut bertanda | null dapat bernilai null.

Kelas	Atribut (modifier: tipe data)	Method publik yang tersedia
User	public id: string; name: string; email: string; passwordHash: string; role: UserRole; status: AccountStatus; lastLoginAt: Date | null; createdAt: Date; updatedAt: Date	constructor(p: Partial<User>); isAdmin(): boolean; isActive(): boolean; toPublic(): object
AuthSession	public id: string; userId: string; tokenHash: string; userAgent: string | null; ipAddress: string | null; expiresAt: Date; revokedAt: Date | null; createdAt: Date	constructor(p: Partial<AuthSession>); isValid(now?: Date): boolean
StudySchedule	public id: string; userId: string; title: string; description: string | null; startTime: Date; endTime: Date; createdAt: Date; updatedAt: Date	constructor(p: Partial<StudySchedule>); durationMinutes(): number
StudySession	public id: string; scheduleId: string; userId: string; status: StudySessionStatus; startedAt: Date | null; completedAt: Date | null; createdAt: Date; updatedAt: Date	constructor(p: Partial<StudySession>); start(now?: Date): void; complete(now?: Date): void; skip(): void
AiSummary	public id: string; userId: string; title: string; sourceContent: string; summaryContent: string; createdAt: Date	constructor(p: Partial<AiSummary>)
SubscriptionPlan	public id: string; code: string; name: string; price: number; currency: string; durationDays: number; isActive: boolean; createdAt: Date	constructor(p: Partial<SubscriptionPlan>); endDateFrom(start: Date): Date
Transaction	public id: string; orderId: string; userId: string; planId: string; amount: number; currency: string; status: TransactionStatus; paymentMethod: string | null; gatewayTransactionId: string | null; paymentUrl: string | null; expiresAt: Date | null; paidAt: Date | null; createdAt: Date; updatedAt: Date	constructor(p: Partial<Transaction>); isPaid(): boolean; isExpired(now?: Date): boolean; amountMatches(gross: number): boolean; markPaid(method: string | null, at?: Date): void

PaymentNotification	public id: string; transactionId: string | null; gatewayEventId: string; payload: Record<string, unknown>; processed: boolean; needsReview: boolean; reviewNote: string | null; receivedAt: Date; processedAt: Date | null	constructor(p: Partial<PaymentNotification>)
Subscription	public id: string; userId: string; planId: string; transactionId: string; status: SubscriptionStatus; startAt: Date; endAt: Date; createdAt: Date	constructor(p: Partial<Subscription>); isActiveAt(now?: Date): boolean
AdminAuditLog	public id: string; adminId: string; targetUserId: string | null; action: string; note: string | null; createdAt: Date	constructor(p: Partial<AdminAuditLog>)
DashboardStats	public totalUsers: number; activeUsers: number; totalSchedules: number; totalSessions: number; completedSessions: number; completionRatePct: number | null	constructor(p: Record<string, unknown>)

Tabel 1. Spesifikasi detail kelas

Kelas	Konstruktor / dependensi	Method utama dan modifier
AuthController	constructor(private service: AuthService)	register, login, refresh, logout, me: public (property arrow function); client: private
AuthService	constructor(private users: UserRepository, private sessions: AuthSessionRepository)	register, login, refresh, logout, verifyAccessToken: public; issueTokens: private
AuthMiddleware	constructor(private auth: AuthService, private users: UserRepository, private subscriptions: SubscriptionService)	authenticate, requireAdmin, requirePremium: public RequestHandler
StudyService	Dependensi repository dan database transaksi	Aturan bisnis untuk jadwal dan sesi belajar
AiSummaryService	Repository ringkasan, layanan langganan, AiProvider	Membuat dan mengambil ringkasan; pembuatan dilindungi pemeriksaan premium pada route
PaymentService	Repository transaksi/paket/notifikasi, SubscriptionService, PaymentGateway	Checkout, daftar transaksi, dan pemrosesan notifikasi pembayaran
SubscriptionService	Repository subscription	Pemeriksaan status premium dan aktivasi langganan
AdminService	Repository admin/user/sesi/notifikasi	Statistik, pengelolaan status user, audit log, dan review notifikasi

Tabel 2. Spesifikasi kelas layanan, controller, dan adapter

1.3 Evaluasi Kelengkapan dan Konsistensi Class Diagram

Dari source code, model domain sudah memuat objek utama untuk akun, sesi autentikasi, jadwal/sesi belajar, ringkasan AI, transaksi, paket premium, langganan, notifikasi pembayaran, audit admin, dan statistik. Pembagian Controller–Service–Repository juga membantu pemisahan tanggung jawab sehingga controller tidak perlu menyusun query database secara langsung.

Temuan konsistensi yang perlu diperhatikan sebelum class diagram dinyatakan final:

- Diagram lapisan .mmd menampilkan kelas ScheduleController, StudySessionController, SubscriptionController, dan ClaudeProvider; implementasi yang tersedia menggunakan StudyController, PaymentController menangani beberapa operasi subscription, dan interface AiProvider. Nama kelas pada diagram perlu diselaraskan dengan file source code.
- Periksa kesesuaian tipe data domain dan database. Di domain, ID direpresentasikan sebagai string dan nilai waktu sebagai Date; harga/amount dikonversi ke number karena kolom numeric dari PostgreSQL dapat diterima sebagai string.
- Atribut nullable harus konsisten dengan skema SQL dan proses bisnis, khususnya lastLoginAt, revokedAt, paymentUrl, paidAt, startedAt, completedAt, serta targetUserId.

Rekomendasi: gunakan source code sebagai acuan implementasi final, lalu sinkronkan diagram .mmd. Pisahkan atribut dari method, tampilkan tipe data dan visibility (+ public, - private, # protected), serta cantumkan kardinalitas relasi. Untuk class yang tidak mempunyai method bisnis, tidak perlu menambahkan method hanya agar diagram tampak lengkap.

BAB II
IMPLEMENTASI KODE PROGRAM

2.1 Implementasi Kelas Entitas / Model

Kelas entitas berada di src/domain/entities.ts. Kelas-kelas tersebut menggabungkan atribut data dan perilaku domain yang berkaitan langsung dengan objek bisnis. Konstruktor menggunakan Partial<T> atau Record untuk mengisi properti dari data yang diperoleh, sedangkan beberapa constructor melakukan konversi tipe numeric agar perhitungan menggunakan number.

```
6  export class User {
7    id!: string; name!: string; email!: string; passwordHash!: string;
8    role!: UserRole; status!: AccountStatus;
9    lastLoginAt!: Date | null; createdAt!: Date; updatedAt!: Date;
10   constructor(p: Partial<User>) { Object.assign(this, p); }
11
12   isAdmin(): boolean { return this.role === UserRole.Admin; }
13   isActive(): boolean { return this.status === AccountStatus.Active; }
14   toPublic() {
15     const { passwordHash: _omit, ...rest } = this;
16     return rest;
17   }
18 }
```

Code 1. src/domain/entities.ts class User

Kelas User merepresentasikan akun pengguna. Method isAdmin() dan isActive() menyederhanakan pemeriksaan role dan status akun. Method toPublic() menghasilkan representasi tanpa passwordHash, sehingga hash kata sandi tidak ikut disertakan pada respons API yang menggunakan method ini. Walaupun atribut passwordHash secara deklarasi masih public, pemisahan representasi publik ini mengurangi risiko kebocoran melalui respons.

```ts
export class StudySession {
  id!: string; scheduleId!: string; userId!: string; status!: StudySessionStatus;
  startedAt!: Date | null; completedAt!: Date | null; createdAt!: Date; updatedAt!: Date;
  constructor(p: Partial<StudySession>) { Object.assign(this, p); }

  start(now = new Date()): void {
    if (this.status !== StudySessionStatus.Planned) throw conflict('Sesi hanya bisa dimulai dari status planned');
    this.status = StudySessionStatus.InProgress;
    this.startedAt = now;
  }
  complete(now = new Date()): void {
    if (this.status !== StudySessionStatus.InProgress) throw conflict('Sesi harus berstatus in_progress untuk diselesaikan');
    this.status = StudySessionStatus.Completed;
    this.completedAt = now;
  }
  skip(): void {
    if (this.status !== StudySessionStatus.Planned) throw conflict('Hanya sesi planned yang bisa dilewati');
    this.status = StudySessionStatus.Skipped;
  }
}
```

Code 2. src/domain/entities.ts class StudySession

Method pada StudySession membatasi perpindahan status agar mengikuti alur yang valid: planned → in_progress → completed, atau planned → skipped. Jika pemanggil mencoba transisi yang tidak diperbolehkan, method melempar conflict error. Aturan ini menjaga konsistensi status sesi belajar.

```ts
export class SubscriptionPlan {
  id!: string; code!: string; name!: string; price!: number; currency!: string;
  durationDays!: number; isActive!: boolean; createdAt!: Date;
  constructor(p: Partial<SubscriptionPlan>) {
    Object.assign(this, p);
    this.price = Number(this.price); // numeric -> string di pg
  }

  endDateFrom(start: Date): Date {
    return new Date(start.getTime() + this.durationDays * 86_400_000);
  }
}
```

Code 3. src/domain/entities.ts class SubscriptionPlan

```
export class Transaction {
  id!: string; orderId!: string; userId!: string; planId!: string;
  amount!: number; currency!: string; status!: TransactionStatus;
  paymentMethod!: string | null; gatewayTransactionId!: string | null; paymentUrl!: string | null;
  expiresAt!: Date | null; paidAt!: Date | null; createdAt!: Date; updatedAt!: Date;
  constructor(p: Partial<Transaction>) {
    Object.assign(this, p);
    this.amount = Number(this.amount);
  }

  isPaid(): boolean { return this.status === TransactionStatus.Paid; }
  isExpired(now = new Date()): boolean {
    return this.status === TransactionStatus.Pending && !!this.expiresAt && this.expiresAt <= now;
  }
  amountMatches(gross: number): boolean { return Math.abs(this.amount - gross) < 0.005; }
  markPaid(method: string | null, at = new Date()): void {
    this.status = TransactionStatus.Paid;
    this.paidAt = at;
    if (method) this.paymentMethod = method;
  }
}
```

Code 4. src/domain/entities.ts class Transaction

Transaction menyimpan informasi pembayaran dan menyediakan pemeriksaan status, masa kedaluwarsa, kesesuaian nominal, serta perubahan ke status paid. Subscription menyediakan isActiveAt(now) yang memeriksa status active dan endAt yang masih berada di masa depan. Dengan demikian, sebagian aturan sederhana diletakkan dekat dengan data yang diaturnya.

Entitas lain melengkapi model sistem: AuthSession memvalidasi refresh session melalui isValid(); StudySchedule menghitung durasi melalui durationMinutes(); SubscriptionPlan menghitung tanggal akhir paket melalui endDateFrom(); AiSummary menyimpan konten sumber dan ringkasan; PaymentNotification merekam notifikasi gateway; AdminAuditLog menyimpan jejak tindakan admin; dan DashboardStats mengubah nilai statistik menjadi number.

2.2 Implementasi Kelas Layanan / Logika Bisnis (*Business Logic*)

Lapisan service berada di src/services. Service menerima permintaan dari controller dan mengorkestrasi operasi lintas entitas, repository, database, serta layanan eksternal. Dengan pembagian ini, validasi aturan bisnis tidak perlu ditulis ulang pada setiap route

Service	Implementasi dan tanggung jawab
AuthService	Registrasi, login, refresh token, logout, dan verifikasi access token. Registrasi memeriksa email yang sudah digunakan, mengenkripsi password menggunakan bcryptjs, dan membuat akun dengan role user.

StudyService	Mengelola pembuatan, pembacaan, perubahan, dan penghapusan jadwal serta transisi sesi belajar. Validasi kepemilikan dan aturan waktu/status diterapkan pada proses bisnis.
AiSummaryService	Mengelola pembuatan dan pembacaan ringkasan AI. Route pembuatan ringkasan menggunakan middleware requirePremium sebelum controller dijalankan.
PaymentService	Menyiapkan checkout, membaca transaksi milik pengguna, dan menangani notifikasi pembayaran. Integrasi gateway dipisahkan melalui PaymentGateway dan implementasi MidtransGateway.
SubscriptionService	Memeriksa apakah pengguna mempunyai akses premium dan mengaktifkan langganan berdasarkan transaksi pembayaran yang valid.
AdminService	Menyediakan statistik dashboard, daftar pengguna, perubahan status akun, audit log, dan notifikasi pembayaran yang memerlukan peninjauan.

Tabel 3. Implementasi kelas layanan

Implementasi AuthService pada register.

```
18    async register(name: string, email: string, password: string): Promise<User> {
19      if (await this.users.findByEmail(email)) throw conflict('Email sudah terdaftar');
20      const hash = await bcrypt.hash(password, 10);
21      return this.users.create(name, email, hash); // role selalu 'user'
22    }
```

Code 5. src/services/AuthService.ts async register

Method register() memeriksa apakah email sudah terdaftar. Jika belum, password di-hash menggunakan bcrypt dengan cost factor 10 sebelum data dikirim ke UserRepository. Password asli tidak disimpan langsung. Role akun baru ditetapkan sebagai user oleh alur repository/service yang ada.

Implementasi AuthService pada login.

```
24  async login(email: string, password: string, client: ClientInfo) {
25    const user = await this.users.findByEmail(email);
26    // compare tetap dijalankan agar waktu respons tidak membocorkan keberadaan email
27    const ok = await bcrypt.compare(password, user?.passwordHash ?? '$2b$10$invalidinvalidinvalidinvalidinvalidinvalidinvalidinvalidi');
28    if (!user || !ok) throw unauthorized('Email atau password salah');
29    if (!user.isActive()) throw forbidden('Akun dinonaktifkan');
30    await this.users.touchLogin(user.id);
31    return { user, ...(await this.issueTokens(user, client)) };
32  }
```

Code 6. src/services/AuthService.ts async login

Method login() memverifikasi password, menolak kredensial yang salah dengan pesan umum, memeriksa status akun, memperbarui waktu login, lalu menerbitkan token. issueTokens() bersifat private: access token dibuat dengan JWT, sedangkan refresh token acak disimpan di database dalam bentuk hash SHA-256 bersama informasi client dan waktu kedaluwarsa. Pada refresh, session lama dicabut sebelum token baru diterbitkan

2.3 Implementasi Kelas Kontroler / Antarmuka Modul Login

Modul login diimplementasikan melalui AuthController pada src/controllers/AuthController.ts, AuthService pada src/services/AuthService.ts, AuthMiddleware pada src/middleware/auth.ts, serta definisi route di src/routes/index.ts. Controller tidak mengimplementasikan proses hash password atau pembuatan token sendiri; tugas tersebut didelegasikan ke AuthService.

Validasi input pada AuthController.

```
5   const registerSchema = z.object({
6     name: z.string().min(1).max(100),
7     email: z.string().email().max(255),
8     password: z.string().min(8).max(72),
9   });
10  const loginSchema = z.object({ email: z.string().email(), password: z.string().min(1) });
11  const tokenSchema = z.object({ refreshToken: z.string().min(1) });
```

Code 7. src/controllers/AuthController.ts const registerSchema

Zod digunakan untuk memvalidasi format data yang masuk. Registrasi memerlukan nama, email yang valid, dan password dengan panjang 8–72 karakter. Login memvalidasi email dan memastikan password tidak kosong. Endpoint refresh/logout memerlukan refreshToken. Jika validasi gagal, error diteruskan ke middleware penanganan error.

Endpoint	Akses	Fungsi dan hasil
POST /auth/register	Publik	Validasi input, membuat akun, mengembalikan status 201 dan user.toPublic().
POST /auth/login	Publik	Memvalidasi kredensial dan mengembalikan profil publik, accessToken, dan refreshToken.
POST /auth/refresh	Publik + refresh token	Memvalidasi refresh token, mencabut session lama, lalu mengembalikan pasangan token baru.
POST /auth/logout	Publik + refresh token	Mencabut session yang terkait dengan refresh token; respons sukses menggunakan status 204.
GET /auth/me	Bearer access token	Memerlukan authenticate middleware dan mengembalikan profil publik pengguna yang sedang login.

Tabel 4. Alur endpoint autentikasi

Middleware autentikasi dan otorisasi diterapkan pada AuthMiddleware.authenticate. AuthMiddleware.authenticate membaca header Authorization dengan format Bearer, memverifikasi access token melalui AuthService, mengambil data pengguna terbaru dari UserRepository, dan menolak

akun yang tidak ditemukan atau berstatus nonaktif. Pengambilan status terbaru penting karena admin dapat menonaktifkan akun setelah token diterbitkan.

requireAdmin memastikan pengguna yang sudah terautentikasi mempunyai role admin. requirePremium memeriksa hak premium melalui SubscriptionService. Contohnya, route POST /summaries memasang authenticate dan requirePremium sebelum AiSummaryController.create. Middleware asyncHandler meneruskan error dari fungsi asynchronous ke mekanisme penanganan error Express.

Implementasi routing modul login.

```
18      // Auth
19      r.post('/auth/register', h(c.auth.register));
20      r.post('/auth/login', h(c.auth.login));
21      r.post('/auth/refresh', h(c.auth.refresh));
22      r.post('/auth/logout', h(c.auth.logout));
23      r.get('/auth/me', authenticate, h(c.auth.me));
```

Code 8. src/routes/index.ts

Route di atas didefinisikan pada src/routes/index.ts. Prefix global API perlu dibaca bersama konfigurasi src/app.ts untuk menentukan URL lengkap saat server berjalan. Daftar route autentikasi menunjukkan pemisahan operasi publik (register/login/refresh/logout) dan operasi yang membutuhkan access token (/auth/me).

LAMPIRAN

https://github.com/maima5/IMPAL-TUGAS.git